

PRIMAL: A Dependency-Free C99 Mathematical Optimization Engine with Unified Conic Modeling and Verifiable Optimization Certificates

Gaetano Minardi
c-vision / Independent Research
Working Paper — October 2026

Abstract. I built PRIMAL to answer a fairly simple question: how far can a general-purpose mathematical optimization solver be taken when it is written from scratch in portable C99 and kept small enough to understand, build and embed without a numerical software stack around it? The result is an open-source solver covering linear and quadratic programming, mixed-integer optimization, second-order cone programming, semidefinite programming, exponential and power cones, and mixed-integer conic models. The implementation uses a shared internal conic representation rather than a collection of unrelated solver front ends. It exposes primal and dual solutions, Farkas rays and recession directions so that an application can check the numerical evidence behind a solver status instead of treating a status code as an assertion of truth. The current repository is dependency-free apart from libm, is implemented in C99, includes 171 runnable examples and 48 ports of public MOSEK examples, and has an automated reliability suite currently reported at 4,706 checks. PRIMAL is not presented as a replacement for mature industrial solvers: the current implementation is slower on several standard benchmarks and still has important dense-KKT and sparse-factorization limitations. The contribution is the engineering combination of broad cone coverage, a small dependency surface, explicit numerical evidence, and reproducible validation. The paper describes the architecture, numerical methods, current results, limitations, and the experiments needed to turn this first research release into a stronger solver.

Keywords: mathematical optimization; conic programming; interior-point methods; C99; numerical linear algebra; optimization certificates; sparse optimization; open-source software; reproducibility.

Repository: <https://github.com/c-vision/Primal>

Paper status: definitive working-paper draft prepared from the public repository state inspected on 3 October 2026. Benchmark numbers are reported as repository results; they are not claimed as a controlled cross-platform performance study.

1. Introduction

Mathematical optimization software is one of those areas where a small amount of mathematics can turn into a very large amount of engineering. The textbook problem may be a few lines of equations; a useful solver has to deal with sparse matrices, scaling, presolve transformations, degeneracy, factorization failures, stopping criteria, infeasibility, unboundedness, model I/O and floating-point arithmetic. Mature optimization products have accumulated this machinery over decades.

PRIMAL started from a different constraint. I wanted to see how much of that stack could be built from scratch in C99 without pulling in a large numerical dependency tree. That constraint is deliberately unfashionable: in a new solver, the obvious way to get performance is to reuse established numerical libraries. I chose not to do that because the implementation itself was part of the experiment.

The result is not a claim that a small new solver is faster than HiGHS, Clarabel, SCS, MOSEK, Gurobi or CPLEX. It is not. Those systems have had years to optimize the parts of the problem where PRIMAL is currently weakest. The interesting question is different: whether a single, inspectable implementation can cover a surprisingly broad set of optimization models, expose enough information to verify its answers, and remain small enough to embed in software that does not want a large solver runtime.

That leads to the central design choice in PRIMAL: supported problem classes are mapped into a common conic representation. LP, QP, SOCP, exponential and power cones, SDP and mixed-integer conic models therefore share parts of the same model and numerical infrastructure. Cone-specific mathematics is still required, but the public surface and much of the solver state do not fragment into separate engines.

The second choice is about trust. An optimization status is useful, but it is not a proof. PRIMAL exposes primal and dual information, complementarity and objective information, and it can return Farkas rays or recession directions. The intention is simple: the caller should be able to check the evidence behind a result. These checks are numerical checks, not formal proofs in exact arithmetic, but they are a substantially better contract than an unchecked status string.

This paper is therefore a report on an implementation and an experiment. It describes what PRIMAL does today, where it works, where it loses to specialized solvers, and which technical limitations are still visible in the code.

2. Research Questions

The project can be stated as five practical research questions:

RQ1 — Coverage. Can one internal architecture cover LP, QP, MIP and several non-polyhedral cones without turning every new model class into a separate solver stack?

RQ2 — Verification. Can the solver return enough primal/dual information for an application to independently check optimality, infeasibility or unboundedness?

RQ3 — Minimality. Can the implementation remain portable C99 with only libc/libm as external runtime requirements while still implementing advanced conic algorithms?

RQ4 — Reproducibility. Can correctness claims be tied to deterministic examples, automated checks and public model instances rather than hand-picked demonstrations?

RQ5 — Practical boundary. Which limitations are caused by the current implementation and which are structural consequences of choosing a small dependency-free architecture?

3. Architecture

The public API is centred on a task/environment model. Model data are converted to a standard internal form using sparse CSC structures for the principal linear and quadratic operators. Presolve removes obvious redundancies and tracks enough information to reconstruct both primal and dual information after the reduced problem has been solved.

The main source modules reflect the numerical pipeline: *presolve.c* and *stdform.c* for model reduction and standardization; *simplex.c* for the two-phase primal simplex; *ipm.c* for LP/QP interior-point methods; *socp.c* for second-order cones; *sdp.c* for semidefinite and unified conic paths; *expcone.c* for exponential and power-cone barrier mathematics; and *linalg.c* for dense/sparse LU, Cholesky and supporting linear algebra.

The architecture is intentionally not purely object-oriented or heavily abstracted. The data structures are close to the mathematics and the C implementation. That makes some optimizations less automatic, but it also makes numerical behaviour easier to inspect. The price is that the project has to solve its own problems in sparse ordering, factorization, regularization and memory management.

Figure 1. PRIMAL solver flow.

Model classes	Common path	Specialized numerical route
LP	standard form → presolve → solver	simplex or Mehrotra IPM; sparse normal equations for large LP
QP / MIQP	standard form → quadratic objective	Mehrotra IPM; branch-and-bound for integer models
SOCp	conic representation	primal-dual IPM with Nesterov-Todd scaling
SDP	PSD cone blocks	primal-dual IPM; $\lambda_{\min}$ tangent-cut fallback
Exp / power cones	native cone blocks	closed-form barrier, Hessian and NT scaling
MICO	conic relaxation	branch-and-bound over conic relaxations

4. Numerical Methods

4.1 Linear and quadratic programming

For LP, PRIMAL combines a two-phase primal simplex with a Mehrotra predictor-corrector interior-point method. Presolve is enabled by default. Large sparse LPs can use normal equations and sparse Cholesky, while smaller systems use dense augmented systems. The simplex implementation uses Dantzig pricing with Bland-style anti-cycling behaviour.

For convex QP, the interior-point route forms the quadratic contribution directly in the Newton system. For large sparse QPs, the repository implements a sparse normal-equation form involving the Hessian and constraint Jacobian. The design deliberately shares as much of the LP infrastructure as possible.

4.2 Mixed-integer optimization

MILP and MIQP are handled by branch-and-bound over continuous relaxations. The current implementation includes SOS branching, semi-continuous and semi-integer variables, bound tightening, probing, strong branching and primal heuristics. MIP feasibility is treated separately from the looser tolerance used inside the tree; candidate incumbents are rechecked against the actual constraints.

This is an important implementation detail rather than a claim of MIP maturity. Industrial MIP engines use substantially more machinery, including sophisticated presolve, cuts, conflict analysis, parallel search and large-scale node management. PRIMAL's MIP route is intended to make mixed-integer conic optimization possible inside the same engine, not to win generic MIP competitions.

4.3 SOCP

The SOCP route is primal-dual and uses Nesterov-Todd scaling for the nonnegative, quadratic and rotated quadratic cones. For larger instances PRIMAL uses a hybrid sparse direction: symmetric normal equations with sparse Cholesky when the scaling point is well behaved, and a sparse augmented-system LU fallback near the cone boundary.

This fallback is currently visible in the benchmark profile of *socp_200*. In the latest investigation, the Newton matrix failed the Cholesky factorization in the final iterations and the solver moved to sparse LU; those fallback solves dominated the remaining solve time. This is useful evidence because it points to a concrete numerical-algebra problem rather than a generic request to 'optimize the solver'.

4.4 SDP and unified conic optimization

The SDP path uses a primal-dual interior-point method with Nesterov-Todd scaling on positive-semidefinite blocks. The unified conic path can assemble PSD blocks together with SOC, rotated-SOC and exponential/power blocks. When the interior-point conversion is not applicable, PRIMAL can use a $\lambda_{\min}$ tangent-cut fallback.

The current implementation has a significant limitation: the unified conic KKT system is assembled densely in the main path. A sparse LDL^T implementation exists and is tested, but is not yet the default factorization for the unified conic system. This is one of the clearest next steps for the project.

4.5 Exponential and power cones

The exponential and power cone implementation contains explicit barrier, gradient and Hessian routines, dual-cone membership and the scaling point used by the primal-dual method. The native route is accepted only when its measured primal/dual residuals and duality gap satisfy the task tolerances; otherwise the dispatcher can fall back to a tangent-cut route. This distinction prevents an attractive-looking iterate from being silently reported as a solved problem.

5. Certificates and Numerical Trust

The part of PRIMAL I consider most important is not the number of model classes. It is the decision to treat verification as part of the solver interface.

For an optimal result, the public API exposes primal and dual quantities from which a caller can reconstruct the usual numerical checks: primal feasibility, dual feasibility, stationarity, complementarity and the primal-dual objective relationship. The exact tolerances are those used by the solver and the model; the checks therefore have to be interpreted in floating-point arithmetic.

For infeasibility, PRIMAL can return Farkas rays. For unboundedness it can return a recession direction. The implementation normalizes the published rays and only publishes them when the corresponding residual conditions measure within the form that was actually solved. If the evidence is not strong enough, the solver can refuse to issue a definitive verdict.

That last behaviour is deliberate. A numerical solver should be allowed to say 'I do not know' when the available evidence does not support a reliable classification. Returning a confident but unverifiable status is worse for an application than returning an explicit numerical failure.

This approach also changes how the test suite is written. A test does not only ask whether an objective has the expected value. It can check the residuals and, where applicable, the certificate itself. This is particularly useful for infeasible and unbounded examples, where there is no ordinary optimal objective to compare.

6. Validation Strategy

The repository contains several layers of validation. The automated reliability suite is currently reported at 4,706 checks. The repository also contains 171 runnable examples and 48 ports of public MOSEK examples. The latter are compatibility tests on the same mathematical instances; they are not evidence that PRIMAL is equivalent to MOSEK as a solver.

The public examples cover LP, QP, MILP, SOCP, SDP, exponential and power-cone models, as well as financial optimization examples. Several are constructed so that the expected optimum is known analytically or can be checked through a second mathematical property such as strong duality, a cone-membership condition or an exact combinatorial optimum.

The repository also includes deterministic larger examples, including sparse transport and flow models, portfolio models, logistic regression with exponential cones, secure beamforming, maximum-clique SDP and other conic formulations.

A particularly useful aspect of this test strategy is that many examples verify the returned solution rather than merely accepting a solver status. This makes the suite closer to a numerical regression test than a collection of smoke tests.

For a publication-quality benchmark release, the remaining step is to freeze the repository commit and publish the raw benchmark output together with compiler, CPU, operating-system, optimization flags and reference-solver versions. The paper therefore treats the numbers below as repository measurements, not as a universal ranking.

7. Experimental Results

7.1 What the current numbers show

The current benchmark results are useful precisely because PRIMAL does not win them all. The reported large LP results show a clear advantage for HiGHS as the problem grows. The SOCP table shows a much larger gap to Clarabel. Those results are consistent with the implementation architecture: PRIMAL's conic KKT path still has denser and less specialized linear algebra than a solver whose main purpose is fast conic optimization.

Family	Problem	PRIMAL	Reference	Interpretation
LP	400 × 200	0.089 s	HiGHS 0.0122 s	PRIMAL slower; gap grows with sparse LP size
QP	200 × 100	0.0443 s	Clarabel faster	Conic/QP path is functional but not performance-leading
SOCP	socp_200	0.0141 s	Clarabel 0.0004 s; SCS 0.0011 s	Sparse KKT/factorization is the current bottleneck
SDP	dimension 22	0.215 s	No direct ranking claimed	Demonstrates a useful SDP size in the current dense path

Table 1. Selected repository benchmark values. These are not normalized cross-platform measurements; the machine, compiler and solver settings must be frozen before making publication-grade speed claims.

7.2 Large sparse LP/QP paths

The repository also reports large sparse paths designed to avoid dense KKT formation. The stated examples include a 90,000-variable transport LP and a 20,000-variable sparse QP. These examples are important because they show that the dependency-free architecture does not automatically imply toy-scale models. However, the reported times should be independently rerun and archived before they are used as headline paper results.

The distinction matters: a solver can be broad without being fast everywhere, and it can be fast on one structured family without being competitive on a different family. PRIMAL's current evidence supports the more modest statement that the implementation has useful sparse paths and can reach non-trivial model sizes.

8. Position Relative to Existing Solvers

PRIMAL is easiest to understand as occupying a specific niche rather than as a universal replacement. HiGHS is the natural open-source reference for large LP/MIP work. Clarabel is a particularly relevant reference for primal-dual conic optimization. SCS covers a broad cone set using a first-order method. MOSEK and the major commercial solvers remain much more mature on industrial-scale workloads.

Capability	PRIMAL	HiGHS	Clarabel	SCS	MOSEK
LP / QP	Yes	Yes	Yes / via cones	Yes	Yes
SOCP	Yes	No	Yes	Yes	Yes
SDP	Yes	No	No	Yes	Yes
Exp / power cones	Native	No	Yes	Yes	Yes
MIP / MIQP	Yes	Yes	No	No	Yes
Core implementation	C99	C++	Rust/C++ ecosystem	C	Proprietary
Runtime dependency goal	libc + libm	larger native stack	native runtime	small C stack	commercial runtime
License	Apache-2.0	MIT	Apache-2.0	MIT	Commercial

Table 2. Capability comparison. The table is deliberately qualitative: it is not a performance ranking. Feature support should be interpreted together with numerical maturity and scale.

The niche is therefore not 'fastest solver'. It is the combination of SDP, exponential/power cones, mixed-integer optimization, primal-dual methods, public certificates and a dependency-free C99 core. That combination is useful for embedded or enterprise applications that want a broad optimization backend without a license server or a large runtime stack.

The 48 MOSEK example ports are best understood as a migration and compatibility aid. They show that the same mathematical problems can be expressed and solved through PRIMAL with matching expected optimal values. They do not establish API or numerical equivalence on arbitrary models.

9. Limitations

The current limitations are not cosmetic and should be part of the paper rather than hidden in a roadmap.

First, the main unified conic KKT system is still assembled densely. That is the clearest ceiling for larger SDP and mixed-cone models. The project already contains a sparse LDL^B implementation, so the next step is integration rather than inventing an entirely new factorization layer.

Second, the SOCP KKT system is not yet expressed in the symmetric form that would allow the sparse symmetric factorization to replace the sparse LU fallback everywhere. The *socp_200* profile makes this limitation visible in practice.

Third, exponential and power-cone polishing is still sensitive to conditioning. Some models may therefore be solved by the tangent-cut fallback even when the native barrier route does not reach the declared relative tolerance.

Fourth, MIP performance is not comparable with mature branch-and-cut engines. The current implementation is valuable because it makes mixed-integer conic models possible inside the same codebase, but it should not be marketed as a generic replacement for HiGHS, Gurobi or CPLEX.

Finally, the benchmark methodology still needs to be frozen as a publication artifact. A serious performance comparison needs identical hardware, pinned compiler settings, repeated runs, clearly defined warm-up rules, raw timing output and a public manifest. The current repository provides the beginnings of that infrastructure; the paper should not pretend that it is already a full benchmark study.

10. What I Would Work on Next

The highest-value work is sparse conic linear algebra, not another long feature list.

1. Integrate sparse LDL into the unified conic path. The implementation already has the factorization and ordering machinery; the missing piece is making the KKT assembly and numerical safeguards use it reliably.
2. Rewrite the SOCP KKT system in a symmetric form. This should reduce the need for sparse LU and directly addresses the failure mode currently visible in *socp_200*.
3. Build a proper benchmark corpus. Generated models are useful for regression testing, but public collections and pathological cases—ill-conditioning, degeneracy, rank deficiency and nearly infeasible cones—are needed for a research evaluation.
4. Separate solver status from verification even more aggressively. A small standalone certificate checker would make the numerical-trust story much stronger because the final verdict would be produced by code independent from the algorithm that generated the candidate.
5. Freeze reproducibility artifacts. Every published number should point to a commit, input hash, compiler, machine description and raw output. This is inexpensive compared with the work already invested in the solver.
6. Improve basis warm starts and the remaining reference-API surface only where those features provide real value. Compatibility is useful, but it should not become an end in itself.

11. Conclusion

PRIMAL began as an engineering question: can a useful mathematical optimization engine be built from scratch in C99 without a large numerical dependency stack? The current answer is yes, with important qualifications.

The implementation already covers a broad set of model classes, including several cones and mixed-integer conic optimization. It has a common internal representation, public primal/dual information, infeasibility and unboundedness certificates, a large reliability suite, runnable examples and compatibility tests against public MOSEK models.

It is also clear where the project is not yet competitive. Large sparse LP is an area where HiGHS is faster. SOCP is an area where Clarabel is dramatically faster on the current benchmark. The dense unified conic KKT system limits scale. These are not embarrassing details; they define the engineering work still required.

The useful contribution of PRIMAL is therefore narrower and, in my view, more interesting: it shows that a single open C99 implementation can bring together LP/QP, MIP, SOCP, SDP and non-symmetric cones while keeping the dependency surface small and treating numerical verification as part of the API. The next stage is not to claim victory over mature solvers. It is to make the numerical core stronger, the experiments more rigorous, and the certificates independently reproducible.

If those steps succeed, PRIMAL can become more than a collection of algorithms. It can become a small, inspectable optimization engine that other software can actually embed—and a useful case study in how much of a modern optimization stack can be built without hiding the mathematics behind a large runtime.

12. Reproducibility Checklist for the Published Version

Artifact	Required action before SSRN submission
Source	Freeze a tagged PRIMAL commit and record the exact hash in the paper.
Compiler	Record compiler name/version, C99 flags and optimization level.
Machine	Record CPU, RAM, OS and relevant numerical-library/runtime details.
Benchmarks	Publish raw output, input sizes and repetition policy; distinguish wall-clock from CPU time.
Reference solvers	Pin versions and settings for HiGHS, Clarabel and SCS where comparisons are made.
Certificates	Publish standalone verification commands and residual tolerances.
Paper	Keep repository, paper and benchmark manifest linked by version/hash.

This checklist is part of the contribution. It prevents the paper from becoming a one-time benchmark screenshot and turns the repository into a reproducible research artifact.

References

- [1] c-vision. PRIMAL: A Dependency-Free Mathematical Optimization Engine. GitHub repository, 2026. <https://github.com/c-vision/Primal>

- [2] Andersen, E. D., and Andersen, K. D. The MOSEK interior point optimizer for linear programming: an implementation of the homogeneous algorithm. In High Performance Optimization, 2000.
- [3] Nesterov, Y., and Todd, M. J. Self-scaled barriers and interior-point methods for convex programming. *Mathematics of Operations Research*, 22(1), 1997.
- [4] O'Donoghue, B., Chu, E., Parikh, N., and Boyd, S. Conic Optimization via Operator Splitting and its Application to Sparse Semidefinite Programs. *Mathematical Programming Computation*, 2016.
- [5] Goulart, P., and Chen, Y. Clarabel: a library for optimization and conic programming. See project documentation and associated publications for the implementation details used in the comparison.
- [6] Huangfu, Q., and Hall, J. A. J. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10, 2018.
- [7] Highs documentation and source repository. <https://highs.dev/>
- [8] MOSEK ApS. MOSEK Optimization Suite documentation, version 11.x. <https://docs.mosek.com/>

Bibliographic note. The final SSRN submission should use the exact editions, DOI metadata and publication years for references [2]–[8]. The URLs above are included so the working paper remains useful; they are not a substitute for a final bibliography pass.

Appendix A. Selected PRIMAL Examples

Example	Model	What it checks
transport	LP	strong duality on a 60×60 transportation model
assignment	LP / integrality	assignment structure and integral optimum
network_flow	LP	flow balance and strong duality
qp_sparse	QP	sparse quadratic objective and duality
knapsack	MILP	integer optimum against exact dynamic programming
facility	MILP	MIP objective against relaxation bound
socp_robust	SOCP	conic feasibility / normal equations
maxcut_sdp	SDP	closed-form SDP optimum
logistic_large	exponential cone	cone feasibility and objective behaviour
secure_ee_sdma	SOCP + exp cone	comparison with a brute-force 2-D true objective
portfolio examples	LP/QP/SOCP/exp/power	financial formulations and independent checks

Appendix B. Scope of the Current Release

The current repository reports support for MPS, LP, OPF and CBF I/O; TASK and PTF remain declared deviations. The public API has grown substantially around solution, parameter, callback, basis and compatibility functions, but numerical coverage and performance remain more important than surface-area parity. The project should therefore continue to treat compatibility claims as testable declarations rather than as a marketing slogan.

Appendix C. Author's Position

This is a first research release. I would rather publish the solver with the benchmark losses and numerical limitations visible than remove them until the graphs look better. The purpose of this paper is to document what exists, make the failure modes reproducible, and make it possible for other people to tell me where the implementation is wrong or incomplete. The benchmark and certificate infrastructure are intended to make that criticism actionable.